

```

import os
import requests
import pandas as pd
import numpy as np
import time
import logging
import threading

from datetime import datetime, timezone, timedelta
from concurrent.futures import ThreadPoolExecutor, as_completed
from http.server import HTTPServer, BaseHTTPRequestHandler

# =====
# Logging Setup
# =====

logging.basicConfig(level=logging.INFO, format="% (asctime)s % (levelname)s % (message)s")
log = logging.getLogger(__name__)

#
# ||  Settings - Edit here only ||
#

TELEGRAM_TOKEN = "8750346745:AAEJBJP_lCr6RLDWr9pj3tvpuRkt6f2tbpg"
TELEGRAM_CHAT_ID = "-1003562604082"
TOP_SYMBOLS_LIMIT = int(os.environ.get("TOP_SYMBOLS_LIMIT", "50"))
PORT = int(os.environ.get("PORT", "8080"))
DEBUG_MODE = os.environ.get("DEBUG_MODE", "false").lower() == "true"

# عدد العملات تُفحص بنفس الوقت
PARALLEL_WORKERS = 8

# ثواني بعد إغلاق الشمعة قبل الفحص
CANDLE_DELAY_SEC = 3

# — Timeframes —
TIMEFRAMES = [
    ("15m", 15, 5, 45, 45),
    ("30m", 30, 10, 90, 60),
    ("1h", 60, 20, 180, 90),
    ("2h", 120, 40, 360, 150),
    ("4h", 240, 80, 720, 270),
    ("1d", 1440, 480, 4320, 1470),
]

# =====
# Core Code - Do NOT edit below (except what's marked)

```

```

# =====
state          = {}
state_lock     = threading.Lock()
cache_lock     = threading.Lock()

# --- Trade History ---
trades_history = []
trades_lock    = threading.Lock()

def save_trade(symbol, frame, price, leverage, lowest_low, drop_pct):
    trade = {
        "time":      datetime.now(timezone.utc),
        "symbol":    symbol,
        "frame":     frame,
        "price":     price,
        "leverage":  leverage,
        "lowest_low": lowest_low,
        "drop_pct":  drop_pct,
    }
    with trades_lock:
        trades_history.append(trade)

def get_trades_report(period: str) -> str:
    now = datetime.now(timezone.utc)
    if period == "today":
        start = now.replace(hour=0, minute=0, second=0, microsecond=0)
        title = "صفقات اليوم 17"
    elif period == "yesterday":
        start = (now - timedelta(days=1)).replace(hour=0, minute=0, second=0, microsecond=0)
        end = now.replace(hour=0, minute=0, second=0, microsecond=0)
        title = "صفقات أمس 17"
    elif period == "week":
        start = now - timedelta(days=7)
        title = "صفقات الأسبوع 17"
    else:
        return "❌ أمر غير معروف"

    with trades_lock:
        if period == "yesterday":
            filtered = [t for t in trades_history if start <= t["time"] < end]
        else:
            filtered = [t for t in trades_history if t["time"] >= start]

    if not filtered:
        return f"{title}\n\nالفترة في هذه صفقات لا توجد."

    lines = [f"{title} ({len(filtered)} صفقة)\n{'-'*30}"]

```

```

for t in filtered:
    lines.append(
        f"🟢 {t['symbol']} | {t['frame']}\n"
        f"السعر: {t['price']:.6f} | رافعة: {t['leverage']}x\n"
        f"أدنى سعر: {t['lowest_low']:.6f} ({t['drop_pct']:.2f}%)\n"
        f"الوقت: {t['time'].strftime('%Y-%m-%d %H:%M')} UTC"
    )
return "\n\n".join(lines)

MEXC_BASE = "https://contract.mexc.com/api/v1/contract"

# — Health Check Server —————
class HealthHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-Type", "text/plain")
        self.end_headers()
        with state_lock:
            active = len(state)
        with trades_lock:
            total = len(trades_history)
        self.wfile.write(
            f"Bot running. Active filters: {active} | Total signals: {total}\n".e
        )
    def log_message(self, format, *args):
        pass

def start_health_server():
    try:
        server = HTTPServer(("0.0.0.0", PORT), HealthHandler)
        t = threading.Thread(target=server.serve_forever, daemon=True)
        t.start()
        log.info(f"Health server on port {PORT}")
    except Exception as e:
        log.warning(f"Health server failed: {e}")

# — Telegram —————
def print_signal(msg):
    print("\n" + "="*60 + "\n" + msg + "\n" + "="*60 + "\n")

def send_telegram(message: str):
    if not TELEGRAM_TOKEN or TELEGRAM_TOKEN == "PUT_YOUR_BOT_TOKEN_HERE":
        return
    try:
        requests.post(
            f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage",
            json={"chat_id": TELEGRAM_CHAT_ID, "text": message, "parse_mode": "HT

```

```

        timeout=15
    ).raise_for_status()
except Exception as e:
    log.error(f"Telegram error: {e}")

# — Telegram Commands Polling —————
_last_update_id = 0

def poll_telegram_commands():
    global _last_update_id
    while True:
        try:
            resp = requests.get(
                f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/getUpdates",
                params={"offset": _last_update_id + 1, "timeout": 30},
                timeout=40
            )
            data = resp.json()
            if not data.get("ok"):
                time.sleep(5)
                continue

            for update in data.get("result", []):
                _last_update_id = update["update_id"]
                msg = update.get("message", {})
                text = msg.get("text", "").strip().lower()
                chat_id = str(msg.get("chat", {}).get("id", ""))

                # المحدد chat السماح فقط لل
                if chat_id != TELEGRAM_CHAT_ID.lstrip("-"):
                    if f"-{chat_id}" != TELEGRAM_CHAT_ID and chat_id != TELEGRAM_
                        continue

                if text in ["/today", "/اليوم"]:
                    report = get_trades_report("today")
                    send_telegram(report)
                elif text in ["/yesterday", "/أمس", "أمس"]:
                    report = get_trades_report("yesterday")
                    send_telegram(report)
                elif text in ["/week", "/الأسبوع", "الاسبوع"]:
                    report = get_trades_report("week")
                    send_telegram(report)
                elif text in ["/status", "/حالة"]:
                    with state_lock:
                        active = len(state)
                    with trades_lock:
                        total = len(trades_history)

```

```

        send_telegram(
            f"🇸🇦 حالة البوت\n"
            f"فلتر نشطة: {active}\n"
            f"إجمالي الإشارات: {total}"
        )
    elif text in ["/help", "/مساعدة"]:
        send_telegram(
            "📁 الأوامر المتاحة:\n\n"
            "/today - صفقات اليوم\n"
            "/yesterday - صفقات أمس\n"
            "/week - صفقات الأسبوع\n"
            "/status - حالة البوت\n"
            "/help - قائمة الأوامر"
        )

except Exception as e:
    log.error(f"Telegram poll error: {e}")
    time.sleep(10)

def start_command_listener():
    t = threading.Thread(target=poll_telegram_commands, daemon=True)
    t.start()
    log.info("Telegram command listener started")

# — Leverage —————
def calc_leverage(entry_price, lowest_low):
    if lowest_low <= 0 or entry_price <= 0 or lowest_low >= entry_price:
        return 40
    drop = (entry_price - lowest_low) / entry_price * 100
    return 40 if drop >= 1.30 else (50 if drop >= 0.85 else 70)

# — MEXC Interval Map —————
def _base_interval(minutes):
    for m, label, bm in [
        (1, "Min1", 1), (5, "Min5", 5), (15, "Min15", 15), (30, "Min30", 30),
        (60, "Min60", 60), (240, "Hour4", 240), (1440, "Day1", 1440), (10080, "Week1", 10080)
    ]:
        if minutes <= m:
            return label, bm
    return "Week1", 10080

# — Data Fetching - Thread-Safe Cache —————
_bar_cache = {}

def get_bars(symbol, minutes, limit=300):
    try:
        base_label, base_min = _base_interval(minutes)

```

```

fetch_limit = min(2000, max(int(limit * (minutes / base_min)) + 50, 300))
cache_key = f"{symbol}_{base_label}_{fetch_limit}"

with cache_lock:
    if cache_key in _bar_cache:
        df = _bar_cache[cache_key].copy()
        if base_min != minutes:
            df = df.resample(f"{minutes}min", origin='start_day',
                             closed='left', label='left').agg(
                {"open": "first", "high": "max", "low": "min",
                 "close": "last", "volume": "sum"}).dropna()
        return df.tail(limit)

end_ts = int(time.time())
start_ts = end_ts - (fetch_limit * base_min * 60)

resp = None
for attempt in range(2):
    try:
        resp = requests.get(
            f"{MEXC_BASE}/kline/{symbol}",
            params={"interval": base_label, "start": start_ts, "end": end_ts,
                    "timeout": 20}
        )
        if resp.status_code == 200:
            break
    except requests.exceptions.RequestException:
        if attempt == 1:
            raise
        time.sleep(1)

if resp is None or resp.status_code != 200:
    return pd.DataFrame()

data = resp.json()
if not data.get("success") or not data.get("data"):
    return pd.DataFrame()

d = data["data"]
if not d.get("time"):
    return pd.DataFrame()

df = pd.DataFrame({
    "open": [float(x) for x in d["open"]],
    "high": [float(x) for x in d["high"]],
    "low": [float(x) for x in d["low"]],
    "close": [float(x) for x in d["close"]],

```

```

        "volume": [float(x) for x in d["vol"]],
    }, index=pd.to_datetime(d["time"], unit="s", utc=True)).sort_index()

    with cache_lock:
        _bar_cache[cache_key] = df.copy()

    if base_min != minutes:
        df = df.resample(f"{minutes}min", origin='start_day',
                        closed='left', label='left').agg(
            {"open": "first", "high": "max", "low": "min",
             "close": "last", "volume": "sum"}).dropna()

    return df.tail(limit)
except Exception as e:
    log.error(f"get_bars {symbol} {minutes}m: {e}")
    return pd.DataFrame()

# — Top Symbols —————
def get_top_symbols(limit=50):
    try:
        resp = requests.get(f"{MEXC_BASE}/ticker", timeout=15)
        data = resp.json()
        if not data.get("success"):
            return []
        tickers = [t for t in data["data"] if t.get("symbol", "").endswith("_USDT")]
        tickers.sort(key=lambda x: float(x.get("volume24", 0) or 0), reverse=True)
        symbols = [t["symbol"] for t in tickers[:limit]]
        log.info(f"Loaded {len(symbols)} symbols")
        return symbols
    except Exception as e:
        log.error(f"get_top_symbols: {e}")
        return []

# — Indicators —————
def ema(s, p): return s.ewm(span=p, adjust=False).mean()
def sma(s, p): return s.rolling(p).mean()

def rma(series, period):
    result = pd.Series(index=series.index, dtype='float64')
    valid = series.dropna()
    if len(valid) < period: return result
    first = valid.index[period - 1]
    result.loc[first] = valid.iloc[:period].mean()
    prev = result.loc[first]
    for i in range(period, len(valid)):
        idx = valid.index[i]
        prev = (prev * (period - 1) + valid.iloc[i]) / period

```

```

        result.loc[idx] = prev
    return result

def calc_smi(df, k=10, d=3, e=10):
    ll = df["low"].rolling(k).min(); hh = df["high"].rolling(k).max()
    dist = df["close"] - (hh+ll)/2; rng = hh - ll
    ds = ema(ema(dist,d),d); rs = ema(ema(rng,d),d)
    val = 200*ds/rs.replace(0,np.nan)
    return val, ema(val.fillna(0),e)

def calc_macd(df, f=12, s=26, sig=9):
    m1 = ema(df["close"],f) - ema(df["close"],s)
    s1 = ema(m1,sig)
    return m1, s1, m1-s1

def calc_rsi(df, p=14):
    d = df["close"].diff()
    rs = rma(d.clip(lower=0),p) / rma(-d.clip(upper=0),p).replace(0,np.nan)
    return 100-(100/(1+rs))

def calc_stoch(df, k=15, ks=3, ds=3):
    ll=df["low"].rolling(k).min(); hh=df["high"].rolling(k).max()
    kk=100*(df["close"]-ll)/(hh-ll).replace(0,np.nan)
    ks_=kk.rolling(ks).mean()
    return ks_, ks_.rolling(ds).mean()

#  1# تعديل - Donchian period: 10 → 20
def calc_donchian(df, p=20):
    return df["close"] > (df["high"].rolling(p).max()+df["low"].rolling(p).min())

# — Filter / Entry / Ongoing / Cancel —————
def check_filter(sym, main_min, conf_min):
    try:
        dm = get_bars(sym, main_min, 200)
        dc = get_bars(sym, conf_min, 100)
        if dm.empty or len(dm)<60: return False
        sv,_=calc_smi(dm)
        if pd.isna(sv.iloc[-1]) or sv.iloc[-1]>-40: return False
        #  2# تعديل - EMA: 40 → 60
        if dm["close"].iloc[-1]>=ema(dm["close"],60).iloc[-1]: return False
        if not calc_donchian(dm).iloc[-1]: return False
        _,_,hm=calc_macd(dm)
        if hm.iloc[-1]>=0: return False
        if dc.empty or len(dc)<30: return False
        _,_,hc=calc_macd(dc)
        if hc.iloc[-1]<=0: return False
        if not calc_donchian(dc).iloc[-1]: return False

```

```

w=dm.loc[dm.index>=dm.index[-1]-pd.Timedelta(days=7)]
if len(w)>=26:
    ml,_,_=calc_macd(w); pos=ml[ml>0]
    if not pos.empty and ml.iloc[-1]>pos.max()*0.20: return False
ml,_,hist=calc_macd(dm)
if ml.iloc[-1]<hist.iloc[-1]: return False
return True
except Exception as e:
    log.error(f"check_filter {sym}: {e}"); return False

def check_entry(sym, entry_min):
    try:
        df=get_bars(sym, entry_min, 100)
        if df.empty or len(df)<30: return False,None
        sv,_,_=calc_smi(df)
        if pd.isna(sv.iloc[-1]) or sv.iloc[-1]>-40: return False,None
        if calc_donchian(df).iloc[-1]: return False,None
        rsi=calc_rsi(df); rsma=sma(rsi,14)
        if pd.isna(rsi.iloc[-1]) or rsi.iloc[-1]>35: return False,None
        com=rsi.dropna().index.intersection(rsma.dropna().index)
        if len(com)<2: return False,None
        if not(rsi.loc[com[-2]]<rsma.loc[com[-2]] and rsi.loc[com[-1]]>=rsma.loc[
k,_,_=calc_stoch(df)
        if pd.isna(k.iloc[-1]) or k.iloc[-1]<=20: return False,None
        return True, df["close"].iloc[-1]
    except Exception as e:
        log.error(f"check_entry {sym}: {e}"); return False,None

def check_ongoing(sym, conf_min):
    try:
        df=get_bars(sym, conf_min, 30)
        if df.empty or len(df)<10: return False
        _,_,h=calc_macd(df)
        return h.iloc[-1]>0 and calc_donchian(df).iloc[-1]
    except: return False

def check_cancel(sym, cancel_min, filter_time):
    try:
        df=get_bars(sym, cancel_min, 20)
        if df.empty: return False
        ft=pd.Timestamp(filter_time)
        if ft.tzinfo is None: ft=ft.tz_localize("UTC")
        recent=df[df.index>ft]
        if recent.empty: return False
        sv,_,_=calc_smi(df)
        for idx in recent.index:
            if idx in sv.index and not pd.isna(sv[idx]) and sv[idx]<=-40:

```

```

        return True
    return False
except: return False

def update_lowest_low(sym, main_min, filter_time, current_low):
    try:
        df=get_bars(sym, main_min, 100)
        if df.empty: return current_low
        ft=pd.Timestamp(filter_time)
        if ft.tzinfo is None: ft=ft.tz_localize("UTC")
        s=df[df.index>=ft]
        return min(s["low"].min(), current_low) if not s.empty else current_low
    except: return current_low

# — Process One Symbol —————
def process_symbol(symbol, now):
    for (label, main_min, entry_min, conf_min, cancel_min) in TIMEFRAMES:
        key = (symbol, label)
        try:
            with state_lock:
                sym_state = state.get(key, {})

            if sym_state.get("filter_time"):
                ft = sym_state["filter_time"]

            new_low = update_lowest_low(symbol, main_min, ft,
                                         sym_state.get("lowest_low", float("in

            with state_lock:
                if key in state:
                    state[key]["lowest_low"] = new_low

            if check_cancel(symbol, cancel_min, ft):
                msg = (f"🚫 Signal Cancelled\n"
                      f"Symbol: {symbol} | Frame: {label}\n"
                      f"Reason: Oversold ({cancel_min}m)")
                print_signal(msg); send_telegram(msg)
                with state_lock: state.pop(key, None)
                continue

            if not check_ongoing(symbol, conf_min):
                with state_lock: state.pop(key, None)
                continue

            entry, price = check_entry(symbol, entry_min)
            if entry and price:
                with state_lock:
                    lowest_low = state.get(key, {}).get("lowest_low", price)

```

```

        if lowest_low == float("inf"): lowest_low = price
        leverage = calc_leverage(price, lowest_low)
        drop_pct = (price - lowest_low) / price * 100 if price > 0 else 0

        # حفظ الصفقة في السجل
        save_trade(symbol, label, price, leverage, lowest_low, drop_p

    msg = (f"🟢 BUY SIGNAL!\n"
           f"Symbol: <b>{symbol}</b>\n"
           f"Frame: <b>{label}</b>\n"
           f"Entry Price: <b>{price:.6f}</b>\n"
           f"Lowest Low: <b>{lowest_low:.6f}</b> ({{drop_pct:.2f}}%"
           f"Suggested Leverage: <b>{leverage}x</b>\n"
           f"Target: Double Capital (2x)\n"
           f"Time: {now.strftime('%Y-%m-%d %H:%M')} UTC")
    print_signal(msg); send_telegram(msg)
    log.info(f"SIGNAL: {symbol} [{{label}}] @ {price} | {leverage}x"
            with state_lock: state.pop(key, None)

    else:
        if check_filter(symbol, main_min, conf_min):
            log.info(f"{symbol} [{{label}}]: ✅ Filter Met")
            send_telegram(f"🔍 Filter Met: {symbol} | {label}")
            df_now = get_bars(symbol, main_min, 5)
            init_low = df_now["low"].iloc[-1] if not df_now.empty else fl
            with state_lock:
                state[key] = {"filter_time": now, "lowest_low": init_low}

    except Exception as e:
        log.error(f"{symbol} [{{label}}]: {e}")

# — Wait Until Next Candle Close —————
def wait_for_next_candle():
    now = datetime.now(timezone.utc)
    secs_left = 60 - now.second - now.microsecond / 1_000_000
    wait = secs_left + CANDLE_DELAY_SEC
    log.info(f"⌚ Waiting {wait:.1f}s for next candle close...")
    time.sleep(wait)

# — Main Loop —————
def main():
    start_health_server()
    start_command_listener()

    now_str = datetime.now(timezone.utc).strftime('%Y-%m-%d %H:%M') + " UTC"
    startup_msg = (
        f"✅ MEXC Futures Bot — التشغيل! \n"

```

```

f"{'-'*30}\n"
f"🇸🇦 الرموز المراقبة : {TOP_SYMBOLS_LIMIT}\n"
f"🕯️ الفريمات : {len(TIMEFRAMES)}\n"
f"⚡ Workers: {PARALLEL_WORKERS}\n"
f"📉 Donchian: 20 | EMA: 60\n"
f"🕒 وقت التشغيل : {now_str}\n"
f"{'-'*30}\n"
f"📋 الأوامر :\n"
f"/today - صفقات اليوم\n"
f"/yesterday - صفقات أمس\n"
f"/week - صفقات الأسبوع\n"
f"/status - حالة البوت\n"
f"/help - قائمة الأوامر"
)
print_signal(startup_msg)
send_telegram(startup_msg)

symbols = get_top_symbols(limit=TOP_SYMBOLS_LIMIT)
if not symbols:
    print("❌ Failed to load symbols."); return

ready_msg = (f"✅ عملة {len(symbols)} تم تحميل\n"
             f"🇸🇦 الفريمات : {len(TIMEFRAMES)}\n"
             f"⚡ Workers: {PARALLEL_WORKERS}\n"
             f"🕯️ مزامنة مع إغلاق الشمعة")
print_signal(ready_msg); send_telegram(ready_msg)

while True:
    try:
        wait_for_next_candle()

        now = datetime.now(timezone.utc)
        log.info(f"🔍 Scanning at {now.strftime('%H:%M:%S')} UTC")

        with cache_lock:
            _bar_cache.clear()

        with ThreadPoolExecutor(max_workers=PARALLEL_WORKERS) as executor:
            futures = {
                executor.submit(process_symbol, sym, now): sym
                for sym in symbols
            }
            for future in as_completed(futures):
                sym = futures[future]
                try:
                    future.result()
                except Exception as e:

```

```
        log.error(f"Worker error {sym}: {e}")

    with state_lock:
        active = len(state)
        log.info(f"✅ Scan complete – Active filters: {active}")

    except KeyboardInterrupt:
        log.info("Bot stopped 🖐️"); break
    except Exception as e:
        log.error(f"Main loop error: {e}"); time.sleep(10)

if __name__ == "__main__":
    main()
```